Terraform on Google Cloud: Lab Notes

Topic: provisioning, modifying and removing a Compute Engine VM with Terraform from Cloud Shell.

Summary

This lab walks through the full lifecycle of a single virtual machine managed as code. A configuration file declares the Google provider and one VM. Terraform first rejects the VM because it has no network attached, which shows that validation happens before anything is created. After the network is added, the VM is built, then changed twice: once with a harmless in-place update (network tags) and once with a change that needs the VM to stop (machine type). The lab ends by removing everything with a single command.

What I set out to learn

- How to install the Terraform CLI in Cloud Shell so it survives between sessions.
- How to point Terraform at a Google Cloud project, region and zone.
- How the plan, apply and destroy commands relate to each other.
- Which kinds of changes Terraform can make in place and which need extra permission in the configuration.

Environment

The lab supplies a temporary Google Cloud project and a student account. Sign in with those credentials only (not a personal account), skip recovery options and free-trial offers, and open Cloud Shell from the console toolbar. Cloud Shell is already authenticated, which can be confirmed with:
gcloud auth list
gcloud config list project

Installing Terraform

Cloud Shell does not ship with Terraform. Writing the install steps into **~/.customize_environment** makes Cloud Shell rerun them every time the session restarts, so the tool is always available. The script adds the HashiCorp apt repository with its signing key and then installs the package:

```
cat <<'EOF' > ~/.customize_environment
wget -O - https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(grep -oP '(?<=UBUNTU_CODENAME=).*' /etc/os-release || lsb_release
-cs) main" | sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update && sudo apt install -y terraform
EOF
```

bash ~/.customize_environment

terraform --version
Any version from 1.5.7 upward works for this lab.

Step 1: Declare the provider

Create an empty **main.tf** with **touch main.tf** and open it in the Cloud Shell Editor. The first block tells Terraform which plugin to download, and the second gives that plugin its default project and location. Replace the three placeholder values with the ones shown in the lab panel.

```
terraform {
  required_providers {
    google = {
      source = "hashicorp/google"
    }
  }
}

provider "google" {
  project = "YOUR_PROJECT_ID"
  region  = "YOUR_REGION"
  zone    = "YOUR_ZONE"
}
```

Running **terraform init** downloads the Google provider and writes **.terraform.lock.hcl**, which pins the provider version so later runs use the same one. This lock file belongs in version control.

Step 2: A VM that Terraform refuses

The first VM definition deliberately leaves out networking:

```
resource "google_compute_instance" "terraform" {
  name         = "terraform"
  machine_type = "e2-micro"

  boot_disk {
    initialize_params {
      image = "IMAGE_PROJECT/IMAGE_FAMILY"
    }
  }
}
```

terraform plan stops with "Insufficient network_interface blocks". The provider schema requires at least one network interface on every Compute Engine instance, so the error appears at planning time and nothing is sent to Google Cloud.

Step 3: Attach a network and build

Add a network interface inside the resource block. The empty **access_config** block asks Google to assign an ephemeral external IP address.

```
  network_interface {
```

```
    network = "default"
    access_config {}
  }
```

Run **terraform plan** again (authorize Cloud Shell if prompted). The plan now lists one resource marked with **+**, meaning it will be created. **terraform apply** shows the same plan and waits for the word **yes** before doing anything. When it finishes, the VM named terraform appears under Compute Engine, VM instances in the console.

Step 4: Change the infrastructure

Adding network tags (in-place update)

Add one line under **machine_type**:

```
  tags = ["web", "dev"]
```

This time the plan marks the resource with **~**, which means Terraform will modify the existing VM rather than replace it. The summary reads 0 to add, 1 to change, 0 to destroy. Confirm with **yes** and the tags are applied while the VM keeps running.

Changing the machine type (needs a stop)

Changing **e2-micro** to **e2-medium** produces a valid plan, but **terraform apply** fails. Google Cloud cannot resize a running VM, and the provider will not stop one unless the configuration explicitly allows it. The fix is one more argument in the resource:

```
resource "google_compute_instance" "terraform" {
  name         = "terraform"
  machine_type = "e2-medium"
  tags         = ["web", "dev"]

  boot_disk {
    initialize_params {
      image = "IMAGE_PROJECT/IMAGE_FAMILY"
    }
  }

  network_interface {
    network = "default"
    access_config {}
  }

  allow_stopping_for_update = true
}
```

With that flag set, plan and apply succeed: Terraform stops the VM, changes its size and starts it again. The error message also mentions **desired_status = "TERMINATED"**, but that leaves the VM stopped after the update, so the flag is the better choice here.

Step 5: Tear everything down

terraform destroy reads the state file, lists every resource it manages with a **-** marker, and removes them after a **yes**. Afterwards the VM list in the console is empty and the project holds nothing created by this configuration.

Command reference

Command	Purpose
terraform init	Downloads providers and prepares the working directory
terraform plan	Compares the configuration with real infrastructure and shows proposed changes without making them
terraform apply	Shows the plan again and carries it out after confirmation
terraform destroy	Removes every resource tracked in the state

Plan symbols

Symbol	Meaning
+	Resource will be created
~	Resource will be updated in place
-	Resource will be destroyed
-/+	Resource will be destroyed and recreated

Takeaways

- Schema errors, such as a missing network interface, are caught during planning, so a quick **terraform plan** after each edit is cheap insurance.
- Reading the plan symbol tells you how disruptive a change will be before you approve it.
- Some updates are legal in Terraform but impossible on a live VM. The provider makes you opt in to downtime with **allow_stopping_for_update** instead of stopping machines on its own.
- Because Terraform tracks what it created, cleanup is one command rather than a hunt through the console.